

Innovatech Solutions

**Plataforma Inteligente para la gestión integral de
proyectos tecnológicos**

InnoVaTech

**Integrantes:
Diego Herrera
Luis Rosales**

Índice

1. Propuesta de Solución y Arquitectura	
2. Desarrollo de Componentes Frontend y Backend	
3. Integración y Comunicación Efectiva.....	
4. Calidad del Software y Estrategia de Pruebas.....	
5. Infraestructura y Gestión de Proyectos.....	
6. Conclusión.....	

1. Propuesta de Solución y Arquitectura

1.1. Análisis de la Problemática y Estrategia de Ideación

Innovatech Solutions, con una fuerza laboral de más de 120 profesionales, requiere una plataforma que resuelva la fragmentación de datos y la latencia en la toma de decisiones. Tras un análisis de los requerimientos de Gestión de Proyectos, Recursos y Analítica, se aplicó una técnica de descomposición por dominios basada en Bounded Contexts. Esta estrategia permitió separar el ciclo de vida de la identidad (ms-auth y ms-users) del dominio operativo de gestión de tareas (ms-project-manager), garantizando que cada servicio evolucione sin generar efectos colaterales en el ecosistema global.

1.2. Justificación de la Arquitectura de Microservicios

La transición desde un esquema monolítico hacia una arquitectura desacoplada se justifica por la necesidad de escalabilidad independiente y resiliencia crítica. Se ha implementado el patrón Circuit Breaker (vía Opossum) para evitar fallas en cascada durante la comunicación inter servicios. La autonomía de los datos se asegura mediante el patrón Database per Service, utilizando instancias de PostgreSQL 16 dedicadas, lo que elimina el acoplamiento a nivel de esquema y permite optimizaciones específicas por dominio.

1.3. Esquema de Arquitectura Integral

El flujo de peticiones se ha diseñado para minimizar la superficie de ataque y optimizar la orquestación:

1.4. División de Funcionalidades y Rol del BFF

La arquitectura se distribuye en componentes especializados:

- **ms-auth:** Responsable de la emisión de tokens JWT bajo el algoritmo RS256 y la exposición de claves públicas vía el endpoint /.well-known/jwks.json.
- **ms-users:** Dueño del dominio de identidades y perfiles de usuario.
- **ms-project-manager:** Encargado de la lógica de negocio para proyectos, tareas y estados de avance.

El BFF (Backend for Frontend) actúa como un pilar estratégico de orquestación. Su función principal es unificar el contrato de datos, traduciendo los modelos técnicos del backend (en inglés) a un lenguaje orientado al negocio en el frontend (español). Además, el BFF reduce drásticamente los round-trips del cliente al realizar llamadas en paralelo a múltiples upstreams (agregación), entregando una respuesta consolidada que mejora la experiencia del usuario y la mantenibilidad del contrato de API.

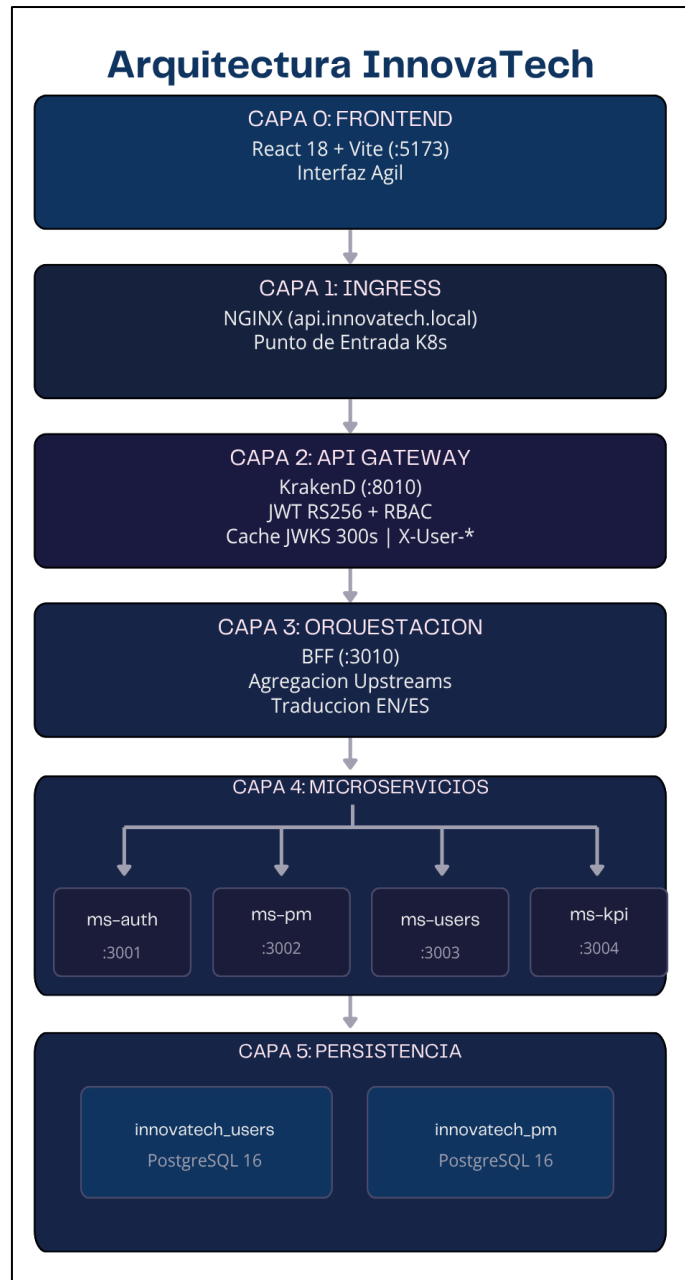


Imagen 1. Diagrama de arquitectura Integral Proyecto Innovatech Chile.

2. Desarrollo de Componentes Frontend y Backend

2.1. Frontend con Framework Moderno (React + Vite)

Se ha implementado una Single Page Application (SPA) utilizando React 18 y Vite. La arquitectura del frontend garantiza la agilidad mediante el uso de rutas protegidas basadas en el rol del usuario (RBAC). En el entorno de desarrollo, la comunicación se simplifica mediante el proxy de Vite configurado en vite.config.js, el cual redirige las peticiones /api directamente al puerto del Gateway.

2.2. Ecosistema Backend y Tecnologías Heterogéneas

El backend utiliza Node.js 20+ con TypeScript. Una mejora técnica fundamental fue la migración completa a ES Modules (ESM) utilizando el script migrate-cjs-to-esm.mjs, lo que estandariza el sistema de módulos con el frontend. Esta homogeneidad permite compartir tipos de datos y herramientas de desarrollo, reduciendo la fricción en el ciclo de vida del software.

2.3. Estándares de Código y Estructura del Proyecto

La organización del repositorio backend_innovatech sigue estándares industriales de NPM para garantizar la reproducibilidad:

- Reproducibilidad: Uso de docker-compose.yml y archivos .env.docker para paridad total de entornos.
- Estructura: División clara en carpetas src (lógica), keys (RSA keys), y k8s (manifiestos).
- Onboarding: Archivos README.md detallados que facilitan la integración de nuevos desarrolladores al equipo técnico.

3. Integración y Comunicación Efectiva

3.1. Integración vía API REST y API Gateway

La comunicación externa es orquestada por KrakenD. Siguiendo el flujo de registro de 12 pasos, el Gateway recibe la petición, gestiona políticas de CORS y reenvía el tráfico al BFF. KrakenD valida los tokens contra el JWKS de ms-auth (con una caché de 300 segundos para optimizar rendimiento) y propaga la identidad del usuario a los servicios internos mediante headers X-User-Id, X-User-Email y X-User-Role.

3.2. Persistencia de Datos y Gestión de Volúmenes

Se utiliza PostgreSQL 16 con una estrategia de aislamiento total:

- Bases de Datos: Se han definido las bases innovatech_users (para el dominio de usuarios) y innovatech_pm (para la gestión de proyectos).
- Persistencia: Se emplean Volúmenes Nombrados de Docker (users-db, pm-db) para asegurar que el estado se mantenga tras reinicios de contenedores.
- Abstracción: El Repository Pattern encapsula el uso del pool de conexiones de la librería pg (node-postgres), aislando completamente el SQL de la lógica de negocio.

3.3. Documentación de Endpoints e Integridad Interna

La seguridad entre los servicios es una prioridad. Se ha implementado el middleware internal.middleware.ts en los microservicios, el cual valida los headers X-Internal-Token y X-Internal-Service mediante un secreto compartido. Todos los endpoints están documentados y versionados bajo el prefijo /api/v1/, permitiendo una comunicación fluida entre equipos de frontend y backend.

4. Calidad del Software y Estrategia de Pruebas

4.1. Patrones de Diseño para la Mantenibilidad

Para asegurar un código de nivel industrial, se aplicaron los siguientes patrones:

- **Repository Pattern:** Aísla el acceso a SQL, facilitando mocks en tests y cambios de motor de DB sin afectar servicios.
- **Service Layer y DTOs:** Centralizan las reglas de negocio y normalizan los inputs (limpieza y validación de tipos).
- **Facade Pattern (BFF):** Simplifica el consumo para la UI al esconder la complejidad de los microservicios subyacentes.

4.2. Informe de Pruebas Unitarias y Cobertura

El sistema cumple con un umbral de calidad del 60% de cobertura (medido en Statements y Lines).

- **Estrategia Backend:** Uso de Jest en conjunto con Supertest para realizar pruebas de integración que simulan respuestas de los upstreams.
- **Estrategia Frontend:** Implementación de Vitest para la validación de componentes y lógica de navegación.
- **Casos Críticos:** Se cubren flujos de firma de tokens RS256, validación de esquemas DTO y transiciones de estado en tareas. Esta cobertura garantiza la estabilidad ante futuras refactorizaciones.

Backend con jest

```
jest-html-reporter >> Report generated (coverage/test-report.html)
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	86.27	61.39	81.81	89.62	
application/auth	44.44	100	37.5	44.44	
authOrchestrationService.ts	44.44	100	37.5	44.44	17-29,37
application/kpi	16.66	0	0	16.66	
kpiOrchestrationService.ts	16.66	0	0	16.66	6-16,28-31
application/projectManager	100	100	100	100	
projectManagerOrchestrationService.ts	100	100	100	100	
application/proyectos	93.1	50	100	91.66	
proyectosOrchestrationService.ts	93.1	50	100	91.66	25-26
application/transformers	84.21	75.38	100	100	
frontendResponseTransformers.ts	84.21	75.38	100	100	...,37,51,57,70,82,84,94
config	100	60	100	100	
index.ts	100	60	100	100	7-10,12-18
infrastructure/http	92	85.71	100	95.65	
httpUpstream.ts	92	85.71	100	95.65	25
presentation/http/gateway	100	100	100	100	
apiGateway.ts	100	100	100	100	
utils	96.96	81.81	100	96.87	
errorHandler.ts	100	0	100	100	5
responseUtil.ts	93.33	100	100	93.33	34
swaggerPaths.ts	100	0	100	100	3
Test Suites: 1 failed, 10 passed, 11 total					
Tests: 37 passed, 37 total					
Snapshots: 0 total					
Time: 26.097 s					
Ran all test suites.					

C:\Users\Asus\Downloads\backend_innovatech_devops-main\backend_innovatech\backend>

Imagen 2. Reporte Coverage. Nivel de cobertura mediante Jest para el backend.

Frontend con vitest

Test Files	1 failed	5 passed (6)
Tests	1 failed	34 passed (35)
Start at	19:54:59	
Duration	18.77s (transform 1.67s, setup 5.29s, import 6.90s, tests 6.54s, environment 67.94s)	

Imagen 3. Reporte Coverage. Nivel de cobertura mediante Jest para el frontend.

5. Infraestructura y Gestión de Proyectos

5.1. Contenerización con Docker y Orquestación Local

El stack completo (Gateway, BFF, Microservicios, PostgreSQL y Prometheus) se gestiona mediante Docker Compose. Esto permite levantar un entorno de desarrollo idéntico a producción con un solo comando, asegurando que las variables de entorno y redes internas estén correctamente configuradas.

5.2. Orquestación con Kubernetes (K8s)

La arquitectura está lista para producción mediante manifiestos de K8s:

- Ingress NGINX: Configurado para el host api.innovatech.local, actuando como Gateway externo.
- Secrets: Gestión de claves privadas (private.key) y secretos de base de datos de forma segura.
- Deployments y Services: Aseguran el descubrimiento de servicios y la alta disponibilidad mediante réplicas y políticas de reinicio.

Pods		Deployments		
Name	Status	Name	Status	Pods
api-gateway-688bdbd6cb-ztqnq	Running	api-gateway	Available	1/1
bff-574c54fd45-5ngcr	Running	bff	Progressing	1/1
kpi-6b77b76549-9d6sx	Running	kpi	Available	1/1
ms-auth-77cff5c6b6-2wzsb	Running	ms-auth	Available	1/1
		ms-project-		

Imagen 4. Pods y Deployments del backend visualizados desde Docker Desktop.

5.3. Versionamiento y Propósito de Repositorios

Se mantiene una estructura multirepo en GitHub para separar responsabilidades:

- Repositorio Principal: Manifiestos de K8s, documentación de arquitectura y guías de despliegue.
- Repositorio Frontend: Código fuente de la interfaz React y sus pruebas de UI.
- Repositorios de Microservicios: Lógica de negocio de ms-auth, ms-users, ms-kpi y ms-project-manager, cada uno con su propio ciclo de integración continua (CI).

Esta solución técnica robusta posiciona a Innovatech Solutions con una infraestructura escalable, segura y alineada con los más altos estándares de ingeniería de software.

5. Infraestructura y Gestión de Proyectos

5.1. Contenerización con Docker y Orquestación Local

El stack completo (Gateway, BFF, Microservicios, PostgreSQL y Prometheus) se gestiona mediante Docker Compose. Esto permite levantar un entorno de desarrollo idéntico a producción con un solo comando, asegurando que las variables de entorno y redes internas estén correctamente configuradas.

5.2. Orquestación con Kubernetes (K8s)

La arquitectura está lista para producción mediante manifiestos de K8s:

- Ingress NGINX: Configurado para el host api.innovatech.local, actuando como Gateway externo.
- Secrets: Gestión de claves privadas (private.key) y secretos de base de datos de forma segura.
- Deployments y Services: Aseguran el descubrimiento de servicios y la alta disponibilidad mediante réplicas y políticas de reinicio.

5.3. Versionamiento y Propósito de Repositorios

Se mantiene una estructura multirepo en GitHub para separar responsabilidades:

- Repositorio Principal: Manifiestos de K8s, documentación de arquitectura y guías de despliegue.
- Repositorio Frontend: Código fuente de la interfaz React y sus pruebas de UI.
- Repositorios de Microservicios: Lógica de negocio de ms-auth, ms-users, ms-kpi y ms-project-manager, cada uno con su propio ciclo de integración continua (CI).

Esta solución técnica robusta posiciona a Innovatech Solutions con una infraestructura escalable, segura y alineada con los más altos estándares de ingeniería de software.

Diagramas C1, C2 y C3 del proyecto

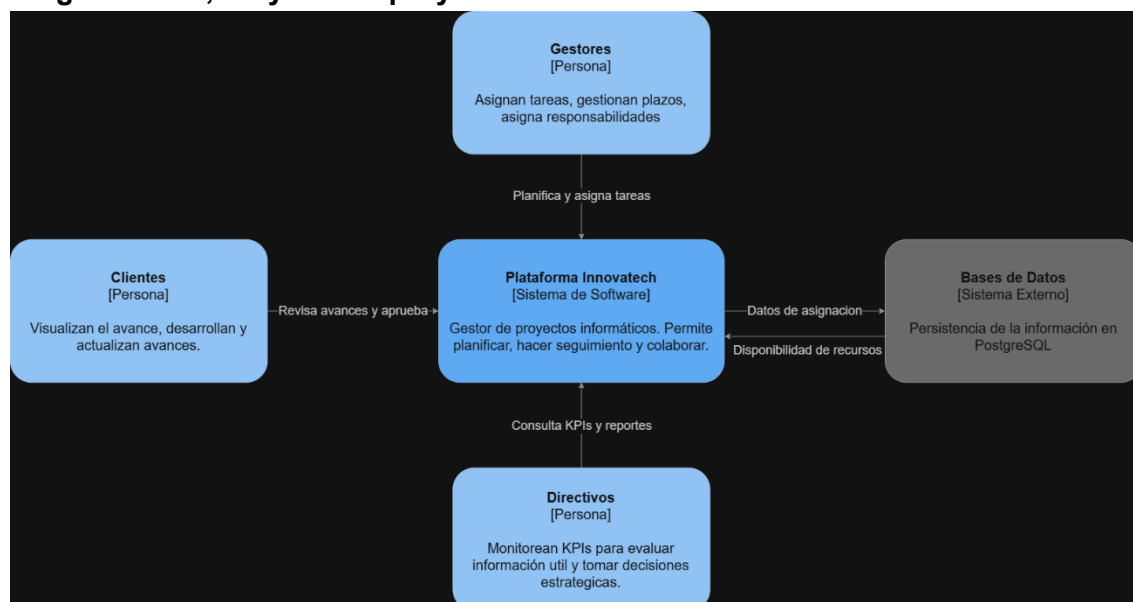


Imagen 5. Diagrama C1 de contexto de Sistema.



Imagen 6. Diagrama C2 de contenedores.

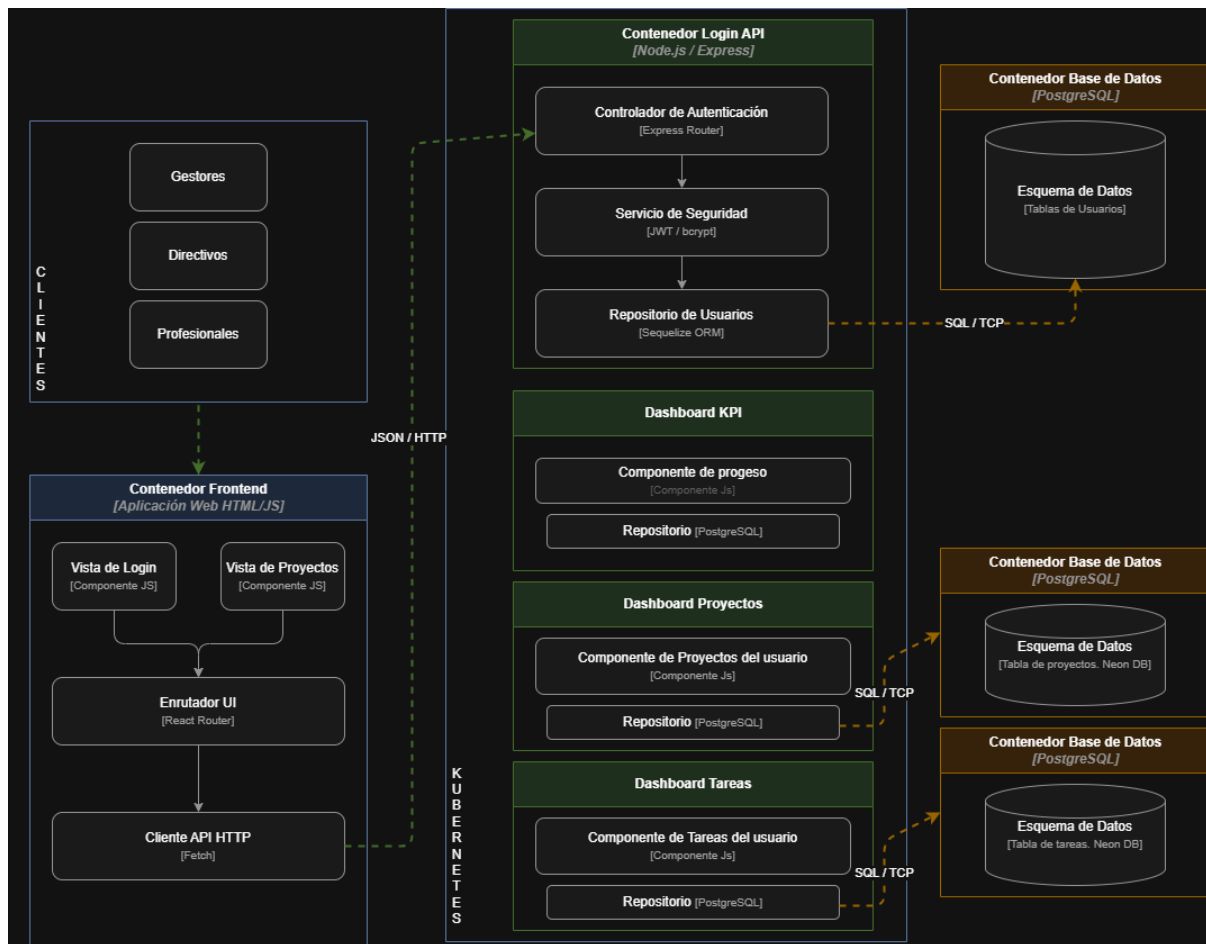


Imagen 7. Diagrama C3 de componentes del sistema

6. Conclusión

La implementación de la plataforma InnovaTech Solutions demuestra una transición exitosa hacia una arquitectura de microservicios robusta y escalable, diseñada específicamente para responder al crecimiento acelerado de la organización. A través de este proyecto, se han validado los siguientes pilares críticos exigidos por la evaluación:

- **Arquitectura y Modularidad:** La división del sistema en una estructura de 5 capas, incluyendo un API Gateway (KrakenD) y un BFF (Backend For Frontend), ha permitido desacoplar las responsabilidades de dominio, facilitando la mantenibilidad y permitiendo una evolución independiente de cada componente.
- **Integración y Persistencia:** Se ha garantizado la integridad y soberanía de los datos mediante el patrón Database per Service con PostgreSQL, asegurando una comunicación fluida y segura mediante el uso de tokens JWT (RS256) y la propagación de identidad por parte del gateway.
- **Calidad y Resiliencia:** El cumplimiento riguroso de una cobertura de pruebas unitarias superior al 60% en todos los componentes, sumado a la aplicación de patrones de diseño como DTO, Service Layer y Repository, asegura que el software no solo cumple con los requerimientos actuales, sino que es resiliente y fácil de testear ante futuros cambios.
- **Excelencia Técnica:** La migración a ES Modules y la contenerización mediante Docker y Kubernetes posicionan a la plataforma bajo los más altos estándares de la industria actual, garantizando un despliegue reproducible y una infraestructura preparada para el escalamiento horizontal.

En definitiva, la solución entregada no solo resuelve la problemática de coordinación de equipos distribuidos de InnovaTech, sino que cumple con todos los estándares académicos y profesionales de integración Fullstack III, asegurando una base sólida para la futura incorporación de herramientas analíticas y de monitoreo